Report primo progetto

INTRODUZIONE AL PROGETTO

Il progetto volge a dimostrare la velocità della programmazione in parallelo rispetto ad una programmazione in serie.

COSA FA IL PROGRAMMA

CREAZIONE DEI DATI

Il programma genera un segnale periodico con rumore "bianco" (all'interno del file **iniz.cpp**).

```
for(int i = 0; i < grand_vet; i++){
    vet[i] = (amp * sin(i/periodo)) + aggiunta_rumore(gen, amp);
}
```

Per riproducibilità è stato fissato un seed di generazione, quindi il rumore sarà sempre lo stesso.

```
mt19937 gen(11);
```

Nello specifico è stata usata una distribuzione uniforme dalla libreria random per creare il rumore (all'interno del file **iniz.cpp**).

```
double ampiezza_rumore = amp / 5.0;
uniform_real_distribution<double> dist(-ampiezza_rumore, ampiezza_rumore);
return dist(gen);
```

CALCOLO SMA

In seguito è stata calcolata una SMA(Simple Moving Avarage) che consiste nel fare la media degli ultimi tot numeri decisi dall'utente (il parmaetro all'interno del programma è **WINDOW**). Questo passo è quello che è stato parallelizzato, all'interno del file: **indicator.cppp** (per **OpenMP**) e **cuda.cu** (per **CUDA**). Il risultato è un segnale con un rumore minore, ma con un offset rispetto a quello iniziale.

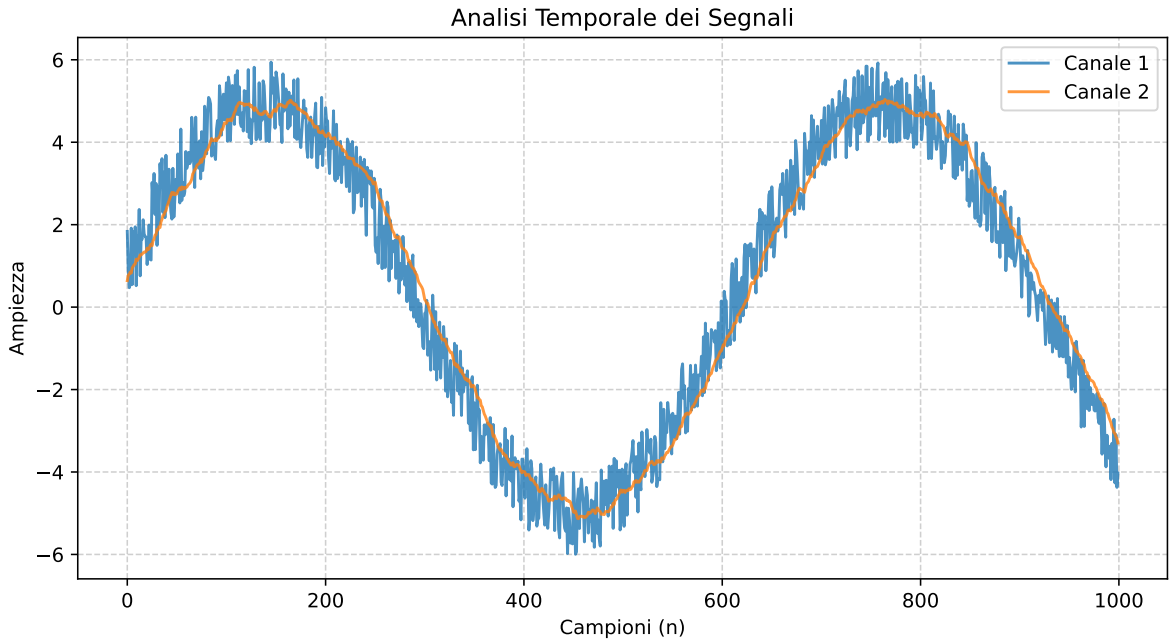


Figure 1: Confronto dei due segnali

COSA SERVE UN SMA

Un ambito conosciuto è il trading, dove con l'aiuto di questo strumento si può diminuire il rumore dei mercati finanziari, aiutando la comprensione di questi.

COME È CALCOLATA

Come accennato sopra è stato fatto in modi diversi. Precisamente sono stati utilizzati tre metodi, due con **OpenMP** e uno per **CUDA**.

SPIEGAZIONE MAIN

```
Inizialmente sono stati salvati i dati in input al main.
if(argc != 8){
    cerr << "Errore! Numero argomenti sbagliato\n";
    return 1;
}

const int GRAND_VET = stoi(argv[1]);
const int WINDOW = stoi(argv[2]);
string func_type = argv[3];
const int THREADS = stoi(argv[4]);
const int REPS = stoi(argv[5]);
const bool STMP = stoi(argv[6]);
string schedule = argv[7];
```

Dopo son stati creati i vettori, quello principale e quello per l'**SMA**. I vettori sono stati creati allocando dinamicamente la memoria perché la grandezza non è fissa. La funzione **stampa** serviva inizialmente per controllare i valori, stampa solo su terminale il vettore appena creato nella funzione **inizializzazione** (spiegata precedentemente).

```
double* vet = new double[GRAND_VET];
inizializzazione(vet, AMP, GRAND_VET, PERIODO);

//stampa(vet, GRAND_VET);

int len = GRAND_VET - WINDOW;
double* sma = new double[len];
```

La parte di CUDA è separata da OpenMP. Si è avuto bisogno di allocare la memoria **cudaMalloc**, copiarla nella GPU **cudaMemcpy**. Poi calcolare i numeri di blocchi da usare con la variabile **blocks** calcolata come $B = \frac{L_{vettore}+N_{thread}-1}{N_{thread}}$. Una volta lanciato il kernel si è fatta la sincronizzazione per misurare il tempo. In fine con **cudaMemcpy** si sono riportati i valori sulla RAM ed infine si è liberata la memoria.

```
if (func_type == "CUDA") {
    double* d_vet = nullptr;
    double* d_sma = nullptr;

    cudaMalloc(&d_vet, GRAND_VET * sizeof(double));
    cudaMalloc(&d_sma, len * sizeof(double));
```

```
    cudaMemcpy(d_vet, vet, GRAND_VET * sizeof(double), cudaMemcpyHostToDevice);

    int blocks = (len + THREADS - 1) / THREADS;

    cout << "Numero: blocchi GPU: " << blocks << endl;

    for (int i = 0; i < REPS; i++) {
        double start = omp_get_wtime();

        lancia_kernel_SMA(d_sma, d_vet, len, WINDOW, blocks, THREADS);

        cudaDeviceSynchronize();
        tempo = omp_get_wtime() - start;

        tempo_tot += tempo;
        if(tempo < temp_min) temp_min = tempo;
        if(tempo > temp_max) temp_max = tempo;
    }

cudaMemcpy(sma, d_sma, len * sizeof(double), cudaMemcpyDeviceToHost);

cudaFree(d_vet);
cudaFree(d_sma);
```

La parte di OpenMP è meno complicata.

```
for (int i = 0; i < REPS; i++) {
    if(func_type == "outer") tempo = SMA_out(sma, vet, len, WINDOW);
    else if(func_type == "intern") tempo = SMA_in(sma, vet, len, WINDOW);

    tempo_tot += tempo;
    if(tempo < temp_min) temp_min = tempo;
    if(tempo > temp_max) temp_max = tempo;
}
```

Infine si sono salvati tutti i dati nei file e stampati su terminale e si è liberata la memoria.

```
long long num_cicli = (long long)(GRAND_VET - WINDOW) * (WINDOW + 1);
cout << "Num. di cicli per singolo test(senza REPS): " << num_cicli << endl;

cout << "Il tempo di esecuzione totale del calcolo dell'SMA e': " << tempo_tot << endl;
```

```
cout << "Il tempo di esecuzione medio del calcolo dell'SMA e': " << tempo_tot/REPS << endl;
cout << "Il tempo min e': " << temp_min << endl;
cout << "Il tempo max e': " << temp_max << endl;

ifstream check_file("dati.csv");
bool file_vuoto = check_file.peek() == ifstream::traits_type::eof();
check_file.close();

ofstream dati_csv("dati.csv", ios::app);

if (!dati_csv.is_open()) {
    cerr << "Il file non è aperto\n";
    return 1;
}

if (file_vuoto) {
    dati_csv << "tipo,schedule,threads,grand_vet,window,num_cili_per_rep,reps,tempo_medio,ter
}

if(STMP == 1){
    dati_csv << func_type << ",";
    dati_csv << schedule << ",";
    dati_csv << THREADS << ",";
    dati_csv << GRAND_VET << ",";
    dati_csv << WINDOW << ",";
    dati_csv << num_cicli << ",";
    dati_csv << REPS << ",";
    dati_csv << tempo_tot/REPS << ",";
    dati_csv << temp_min << ",";
    dati_csv << temp_max << endl;
}

//stampa(sma, len);

ofstream out_csv("output.csv");

if (!out_csv.is_open()) {
    cerr << "Il file non è aperto\n";
    return 1;
}

for(int i = WINDOW; i < STAMPA + WINDOW; i++){
```

```
    out_csv << vet[i] << ",";
    out_csv << sma[i - WINDOW];
    out_csv << endl;
}

out_csv.close();

delete[] vet;
delete[] sma;

cout << "Programma Finito!\n";

return 0;
```

Ciclo esterno (OpenMP) Funzione

Nel **Makefile** è stato nominato **outer**, indica che la parallelizzazione è stata fatta sul ciclo esterno quindi quello più grande.

```
double start = omp_get_wtime();

#pragma omp parallel for schedule(runtime)
for(int i = 0; i < len; i++){
    double sum = 0;
    int index = i + window - 1;
    for(int j = index; j >= i; j--){
        sum += vet[j];
    }
    sma[i] = sum / window;
}
double stop = omp_get_wtime();
return stop - start;
```

Ciclo interno (OpenMP) Funzione

Nel **Makefile** è stato nominato **intern**, indica che la parallelizzazione è stata fatta sul ciclo interno quindi quello più piccolo.

```
double start = omp_get_wtime();

for(int i = 0; i < len; i++){
```

```
    double sum = 0;
    int index = i + window - 1;

    #pragma omp parallel for reduction(+:sum) schedule(runtime)
    for(int j = index; j >= i; j--){
        sum += vet[j];
    }

    sma[i] = sum / window;
}

double stop = omp_get_wtime();
return stop - start;
```

CUDA Funzione

Nel **Makefile** è stato nominato **CUDA**.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;

if (i < len) {
    double sum = 0.0;
    int index_fine = i + window - 1;
    for (int j = index_fine; j >= i; j--) {
        sum += vet[j];
    }
    sma[i] = sum / (double)window;
}
```

DATI RACCOLTI

I dati sono stati inseriti in un file denominato **dati.csv** e riportati sottostante.

Table 1: Confronto performance (tempi in secondi)

tipo	schedule	threads	grand_vet	window	num_cili_per_rep	reps	tempo_medio	tempo_min
outer	static	1	32768	256	8355584	100	0.015942	0.015137
outer	static	1	65536	256	16776960	100	0.033356	0.030569
outer	static	1	131072	256	33619712	100	0.063262	0.061352
outer	static	1	262144	256	67305216	100	0.128265	0.123352
outer	static	1	524288	256	134676224	100	0.253735	0.247577

tipo	schedule	threads	grand_vet	window	num_cili_per_rep	reps	tempo_medio	tempo_min
outer	static	1	1048576	256	269418240	100	0.505159	0.496189
outer	static	1	2097152	256	538902272	100	1.005650	0.995683
outer	static	1	4194304	256	1077870336	10	2.014150	2.000230
outer	static	1	8388608	256	2155806464	10	4.040840	4.024380
outer	static	1	2097152	512	1075576320	100	2.109970	2.090850
intern	static	1	2097152	512	1075576320	100	2.539430	2.514890
outer	static	2	2097152	512	1075576320	100	1.064140	1.052330
outer	dynamic	2	2097152	512	1075576320	100	1.106040	1.095550
outer	guided	2	2097152	512	1075576320	100	1.063170	1.052940
intern	static	2	2097152	512	1075576320	50	2.344670	2.313780
intern	dynamic	2	2097152	512	1075576320	10	19.758700	19.619200
intern	guided	2	2097152	512	1075576320	50	3.029110	2.994520
outer	static	4	2097152	512	1075576320	100	0.548702	0.543297
outer	dynamic	4	2097152	512	1075576320	100	0.582503	0.577831
outer	guided	4	2097152	512	1075576320	100	0.547042	0.540269
intern	static	4	2097152	512	1075576320	50	2.562440	2.531670
intern	dynamic	4	2097152	512	1075576320	10	20.645800	20.594200
intern	guided	4	2097152	512	1075576320	25	6.227860	6.175580
outer	static	8	2097152	512	1075576320	100	0.287185	0.283874
outer	dynamic	8	2097152	512	1075576320	100	0.317303	0.305797
outer	guided	8	2097152	512	1075576320	100	0.302677	0.296957
intern	static	8	2097152	512	1075576320	25	3.698810	3.640610
intern	dynamic	8	2097152	512	1075576320	25	25.377100	25.114300
intern	guided	8	2097152	512	1075576320	25	10.670600	10.546800
outer	static	16	2097152	512	1075576320	100	0.219975	0.186041
outer	guided	16	2097152	512	1075576320	100	0.239360	0.226788
outer	dynamic	16	2097152	512	1075576320	100	0.226500	0.203834
outer	static	32	2097152	512	1075576320	100	0.227711	0.201904
outer	dynamic	32	2097152	512	1075576320	100	0.228785	0.217975
outer	guided	32	2097152	512	1075576320	100	0.233453	0.208749
outer	static	64	2097152	512	1075576320	100	0.223078	0.208712
outer	dynamic	64	2097152	512	1075576320	100	0.234275	0.216850
outer	guided	64	2097152	512	1075576320	100	0.221540	0.205252
outer	static	128	2097152	512	1075576320	100	0.215222	0.204685
outer	dynamic	128	2097152	512	1075576320	100	0.230276	0.221701
outer	guided	128	2097152	512	1075576320	100	0.221071	0.204414
CUDA	nan	32	2097152	512	1075576320	100	0.006883	0.006769
CUDA	nan	64	2097152	512	1075576320	100	0.006877	0.006743
CUDA	nan	128	2097152	512	1075576320	100	0.006848	0.006721
CUDA	nan	256	2097152	512	1075576320	100	0.006821	0.006723
CUDA	nan	512	2097152	512	1075576320	100	0.006831	0.006708

tipo	schedule	threads	grand_vet	window	num_cili_per_rep	reps	tempo_medio	tempo_min
CUDA	nan	1024	2097152	512	1075576320	100	0.006864	0.006758
CUDA	nan	32	33554432	512	17213160960	100	0.107653	0.107433
CUDA	nan	64	33554432	512	17213160960	100	0.107665	0.107322
CUDA	nan	128	33554432	512	17213160960	100	0.107614	0.107438
CUDA	nan	256	33554432	512	17213160960	100	0.107643	0.107251
CUDA	nan	512	33554432	512	17213160960	100	0.107329	0.107142
CUDA	nan	1024	33554432	512	17213160960	100	0.107885	0.107715

Tabella dati

DIFFICOLTA' DEL PROBLEMA

Come si può notare dal grafico sottostante la difficoltà cresce linearmente con la grandezza del vettore

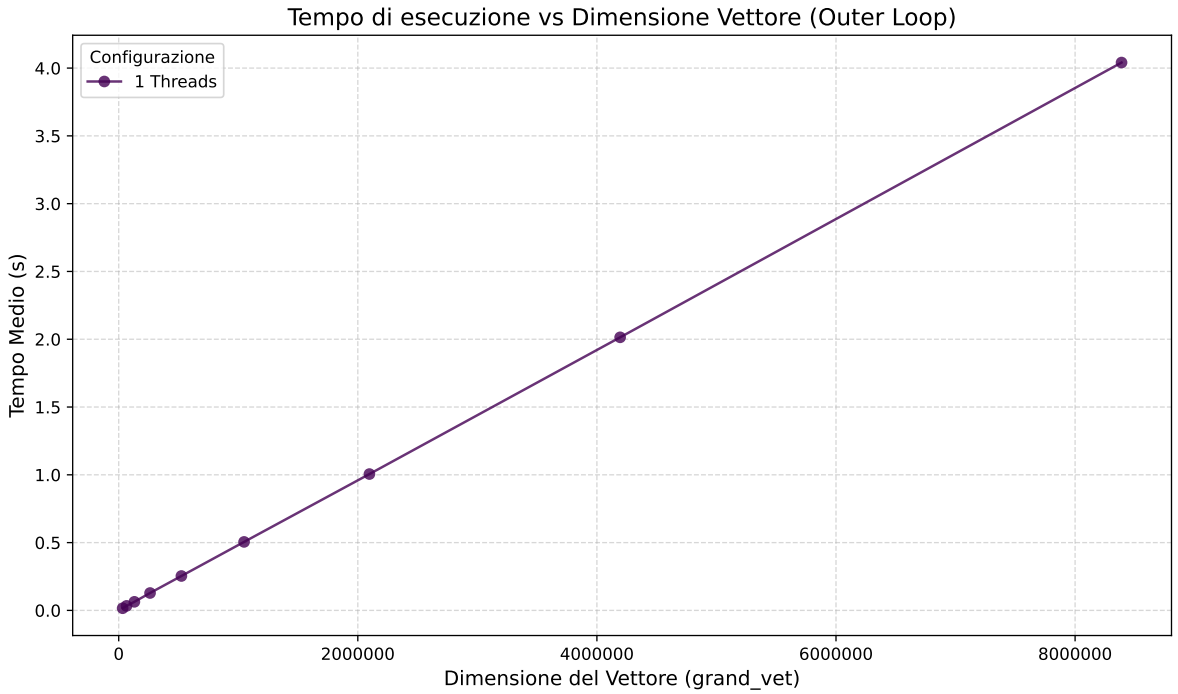


Figure 2: Analisi della scalabilità rispetto alla dimensione del vettore

Se si aumenta la grandezza della finestra di calcolo dell'SMA, cresce anche la difficoltà del problema in quanto aumentano i cicli da dover eseguire

SPEED-UP

Lo **Speed-up** è calcolato come:

$$S_{up} = \frac{S_{base}}{S_{calc}}$$

Indica quanto veloce è laparallelizzazione rispetto al programma eseguito sequenzialmente.

Ciclo esterno

Table 2: Speed-up delle implementazioni Outer

	Tipo	Threads	Schedule	Tempo Medio (s)	Speed-up
9	outer	1	static	2.1100	1.00
11	outer	2	static	1.0641	1.98
12	outer	2	dynamic	1.1060	1.91
13	outer	2	guided	1.0632	1.98
17	outer	4	static	0.5487	3.85
18	outer	4	dynamic	0.5825	3.62
19	outer	4	guided	0.5470	3.86
23	outer	8	static	0.2872	7.35
24	outer	8	dynamic	0.3173	6.65
25	outer	8	guided	0.3027	6.97
29	outer	16	static	0.2200	9.59
30	outer	16	guided	0.2394	8.82
31	outer	16	dynamic	0.2265	9.32
32	outer	32	static	0.2277	9.27
33	outer	32	dynamic	0.2288	9.22
34	outer	32	guided	0.2335	9.04
35	outer	64	static	0.2231	9.46
36	outer	64	dynamic	0.2343	9.01
37	outer	64	guided	0.2215	9.52
38	outer	128	static	0.2152	9.80
39	outer	128	dynamic	0.2303	9.16
40	outer	128	guided	0.2211	9.54

Da questa tabella si nota che il test migliore ha ottenuto uno **Speed-up** di 9.80x, utilizzando lo schedule static con 128 threads.

Inizialmente lo Speed-up è circa due volte superiore a quello del numero di Thread inferiore quindi un andamento lineare.

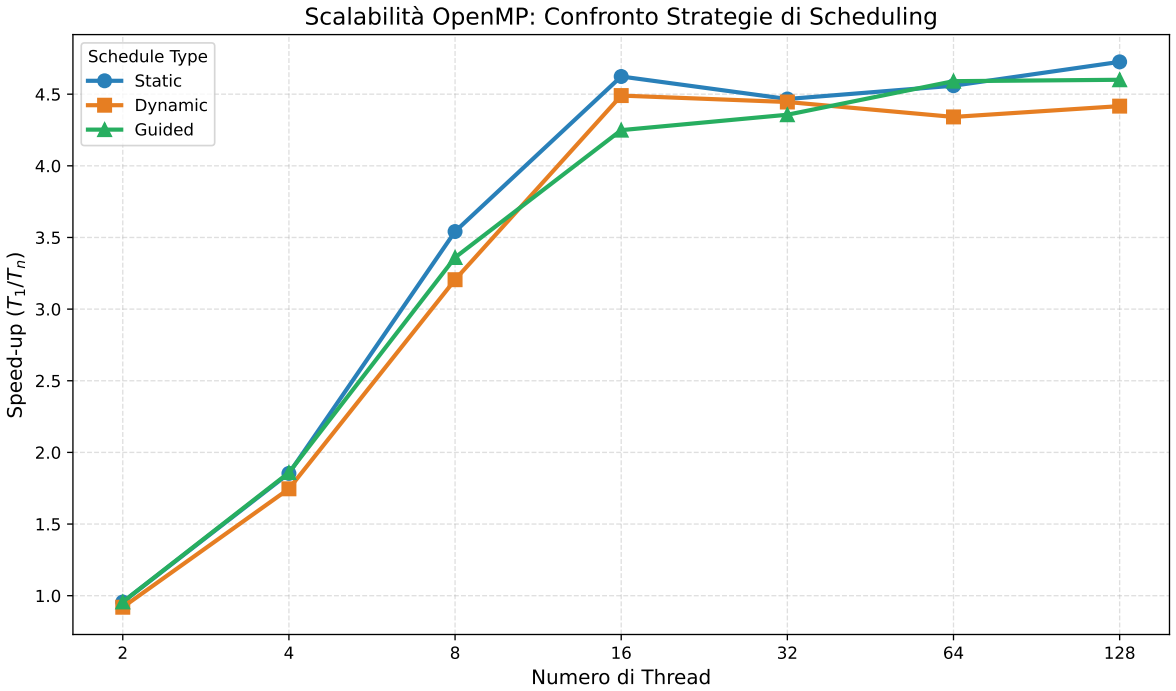


Figure 3: Analisi della scalabilità: confronto Static, Dynamic e Guided (Outer)

Come si può notare fino a 16 Thread la velocità ha continuato a salire, anche se da 8 a 16 non è raddoppiata, poi si è stabilizzata. Questo fenomeno è dovuto al fatto che il processore utilizzato è un **Intel(R) Core(TM) Ultra 7 255H** che ha 16 core fisici ed un thread per core, quindi nel momento in cui vengono superati i "limiti fisici", non si notano più grandi miglioramenti.

Confronto tra scheduling

Tra gli scheduling analizzati, lo static risulta generalmente molto efficiente perché assegna il carico computazionale ai thread in fase iniziale, riducendo l'overhead di gestione. Tuttavia, dai risultati sperimentali emerge che non esiste una scelta universalmente migliore: in alcuni casi lo static ottiene le migliori prestazioni, mentre in altri lo scheduling guided risulta più efficiente. Questo comportamento dipende dal numero di thread e dalla dimensione del problema, che influenzano il bilanciamento tra overhead di assegnazione e bilanciamento del carico.

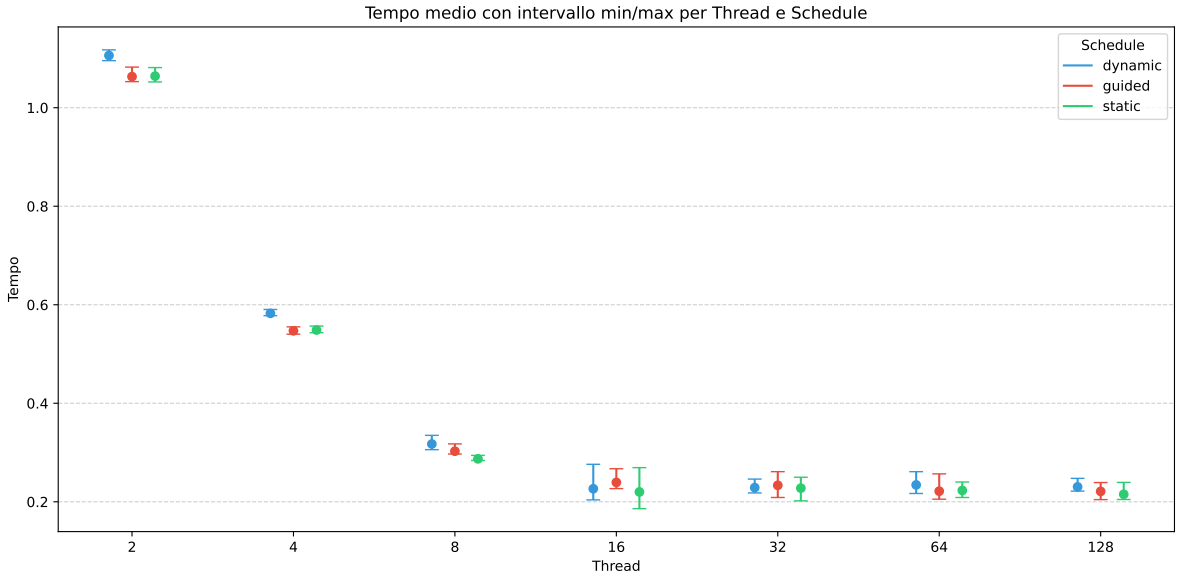


Figure 4: Tempo medio con min e max per Thread e Schedule

Ciclo interno

Analizzando la parallelizzazione sul ciclo interno di soli 512 valori alla volta si nota che più thread vengono usati più peggiora la situazione.

Table 3: Speed-up delle implementazioni Intern

	Tipo	Threads	Schedule	Tempo Medio (s)	Speed-up
10	intern	1	static	2.5394	1.00
14	intern	2	static	2.3447	1.08
15	intern	2	dynamic	19.7587	0.13
16	intern	2	guided	3.0291	0.84
20	intern	4	static	2.5624	0.99
21	intern	4	dynamic	20.6458	0.12
22	intern	4	guided	6.2279	0.41
26	intern	8	static	3.6988	0.69
27	intern	8	dynamic	25.3771	0.10
28	intern	8	guided	10.6706	0.24

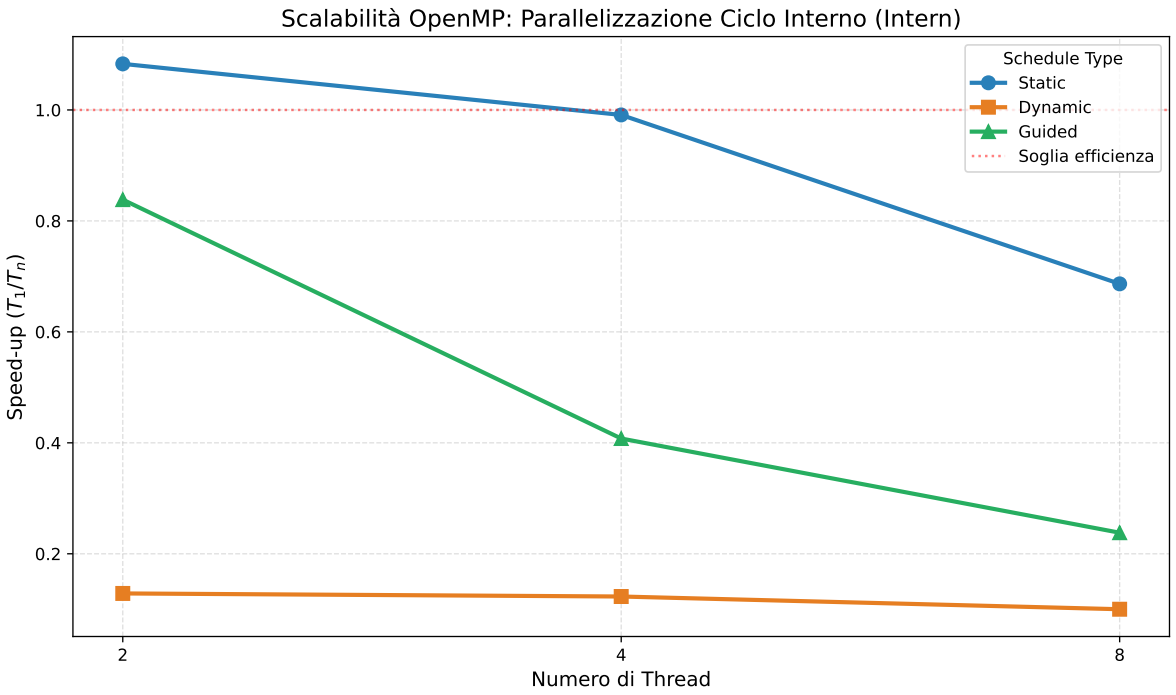


Figure 5: Analisi della scalabilità: confronto Static, Dynamic e Guided (Intern)

Dai grafici si nota un calo importante delle prestazioni, probabilmente dovuto al fatto che i pochi cicli parallelizzati rendono inefficiente il lavoro. Il richiamo dei thread occupa più spazio che il miglioramento nelle prestazione. Quindi risulta inutile parallelizzare il ciclo interno.

Si può anche notare una grande differenza tra i vari schedule, dove lo static domina rispetto agli altri.

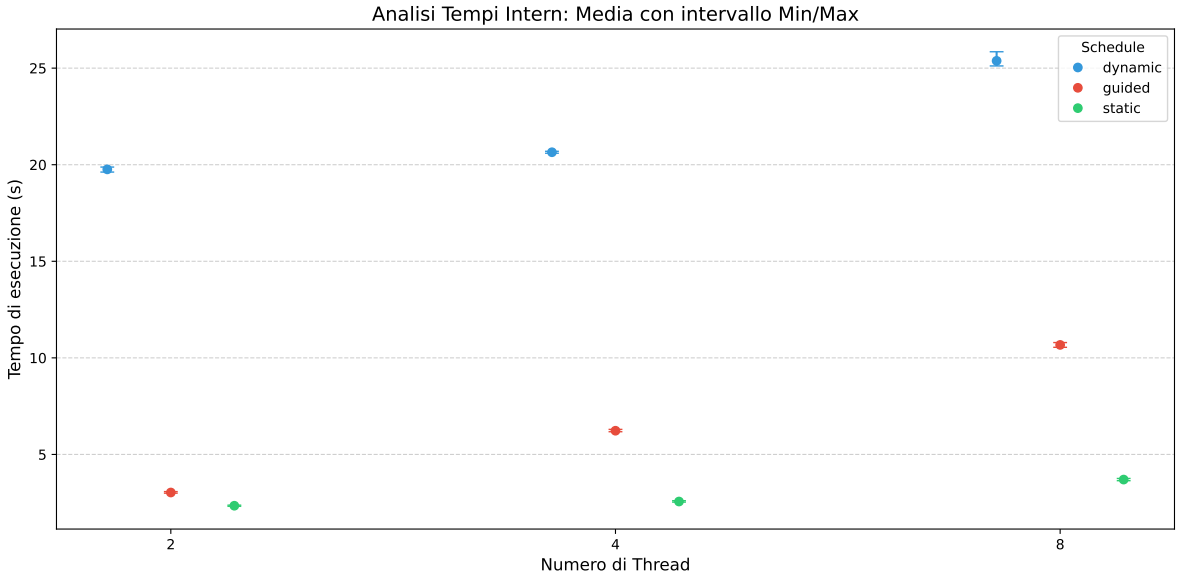


Figure 6: Tempo medio con min e max per Thread e Schedule (Intern)

Nota

Non sono stati messi i thread superiori, perché ci metteva troppo a realizzare un singolo test, anche più di dieci minuti.

CUDA

CUDA utilizza la scheda grafica per fare i calcoli ed è molto potente.

Table 4: Speed-up CUDA rispetto a CPU Sequenziale (Outer 1 Thread)

	Tipo	Thread per blocco	Tempo Medio (s)	Speed-up (Vs CPU)
41	CUDA	32	0.006883	306.57
42	CUDA	64	0.006877	306.79
43	CUDA	128	0.006848	308.12
44	CUDA	256	0.006821	309.35
45	CUDA	512	0.006831	308.88
46	CUDA	1024	0.006864	307.41

Analizzando i dati raccolti, si nota che il test migliore utilizzando CUDA ha ottenuto uno Speed-up di 309.35x rispetto all'esecuzione sequenziale su CPU, configurando il kernel con 256 threads per blocco.

Ipotesi

Tra i 32 e i 1024 threads per blocco non ci sono grandi differenze, questo probabilmente è dovuto ad un problema di **Memory Bound** in cui dovendo richiamare 512 valori, la VRAM fa fatica. Questo si potrebbe risolvere con una **Shared Memory** in modo da non dover richiamare ogni volta i dati dalla memoria.

CONFRONTO TRA I VARI METODI

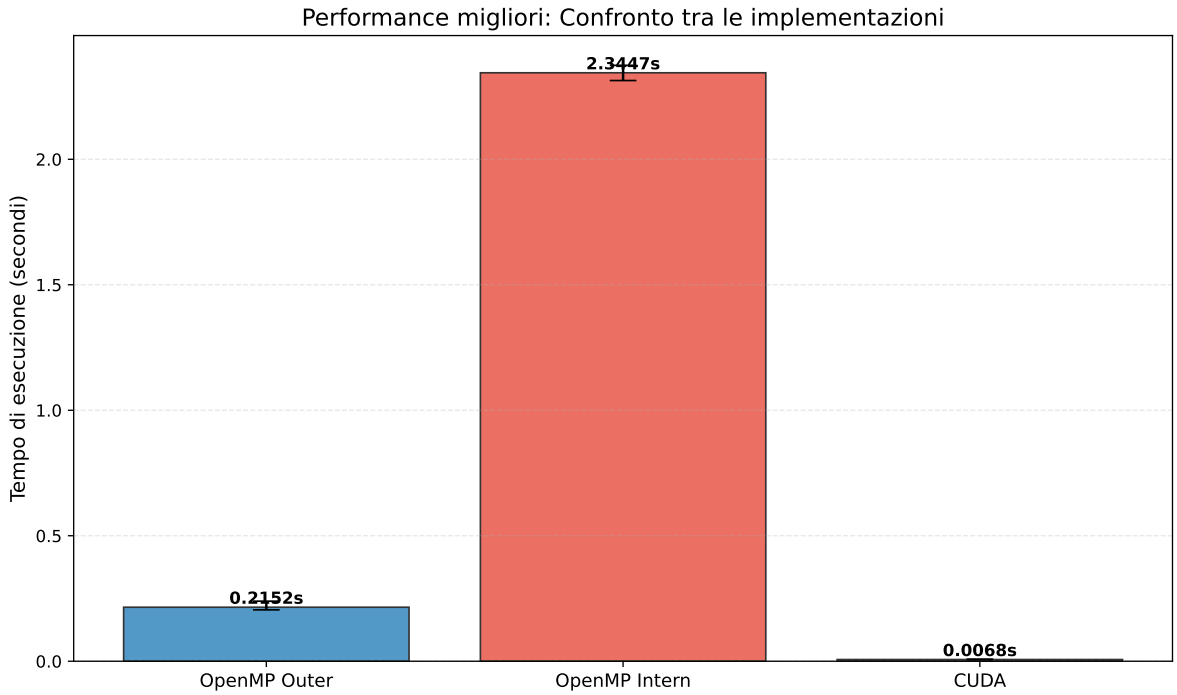


Figure 7: Confronto dei tempi migliori: OpenMP (Outer vs Intern) e CUDA